Project Name: Project 1: Voting System **Team#01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: STV_Shuffle_1

Name(s) of Testers: Horoom Bula

Test Description: Test STV voting with 4 candidates and 2 seats and shuffle.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __x_

Results: Pass _X____ Fail_____

Preconditions for Test:
There will be STV Ballots with four candidates and atleast 2 ballots.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Put test data in Test.csv where each line is an ID followed by candidate preferences	Ballot(1,(1,2,3,4)), Ballot(2,(2,1,3,4)), Ballot(3,(3,4,2,1)), Ballot(4,(1,3,2,4)), Ballot(5,(4,2,1,3))	File is correctly structured with unique IDs and consistent list lengths	File read successfully	
2	Run the main program	main() execution	Program prompts for inputs as expected		

3	Fill out inputs according to test description and press Enter after each	4 candidates, 2 seats, STV shuffle test	Input accepted without errors		
4	Look at Election_Report.txt and verify election summary		Matches expected results below	Matches expected results	
			Expected Output: Total Candidates: 4 Seats to Fill: 2 Candidates: - Alice - Bob - Charlie - Diana Elected: * Alice * Bob Non-Elected: - Charlie - Diana		

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team#01**

Test Stage: Unit __ System _X_ **Test Date: November 2nd**

Test Case ID#: Plurality_1

Name(s) of Testers: Horoom Bula

Test Description: Test plurality voting with 4 candidates and 1 seats and no shuffle.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __x_

Results: Pass __X___ Fail___

Preconditions for Test:
There will be plurality with some 4 ballots and one seat as an option.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input test data into PTest.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,0,0,0)), Ballot(2,(0,1,0,0)), Ballot(3,(0,0,0,1)), Ballot(4,(1,0,0,0)), Ballot(5,(0,0,1,0))	File is correctly structured with unique IDs and consistent list lengths	File read successfully	Plurality_1
2	Run main	main() execution	Program prompts for inputs as expected	Program ran successfully	Plurality_1
3	Fill out inputs according to test description and press Enter after each	4 candidates, 1 seat, no shuffle	Input accepted without errors	Input accepted correctly	Plurality_1

4	Look at Election_Report.txt and verify election summary	N/A	Elected: * Alice (Votes: 40%) Non-Elected: - Bob (Votes: 20%) - Charlie (Votes: 20%) - Diana (Votes: 20%)	Elected: * Alice (Votes: 2) Non-Elected: - Bob (Votes: 1) - Charlie (Votes: 1) - Diana (Votes: 1)	Semi-failed

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team#01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: PL_Tie

Name(s) of Testers: Horoom Bula

Test Description:
There will be a tie for one seat available for plurality.

Automated: yes___ no __x_

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Ptest.csv

Results: Pass ___X__ Fail_____

Preconditions for Test:
There will be an csv before the test where there is a tie for first, and line 26 of main must have Ptest.csv as well as line 42

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1					

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Put the test data into PTest.csv file	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,0,0,0)), Ballot(2,(0,1,0,0)), Ballot(3,(1,0,0,0)), Ballot(4,(0,0,1,0)), Ballot(5,(0,1,0,0))	Candidate 1 and 2 should win at random	Candidate 1 always wins	Plurality tie test
2	Run the main program	main() execution	Program runs and prompts for inputs	Program ran successfully	
3	Input necessary data (plurality vote, 2 seats, no shuffle)	N/A	Inputs accepted	Inputs accepted	
4	Check results in Election_Report.txt	N/A	Election summary reflects expected results	Election summary shows Candidate 1 always wins	Repeat steps 1-4 to test randomness

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team#01**

Test Stage: Unit __ System _X_ **Test Date: November 2nd**

Test Case ID#: STV_One_Ballot **Name(s) of Testers: Surafiel Ghirmai**

Test Description:

This is a test with STV in one ballot with more than one seat to test tiebreakers

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __x_

Test

Results: Pass ___X__ Fail_____

Preconditions for Test:
There will be an csv before the test with one ballot, and line 26 of main must have test.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input test data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4))	One guaranteed winner for candidate 1 while others are random	One guaranteed winner for candidate 1 while others are random	STV one ballot test
2	Run the main program	main() execution	Program runs and prompts for inputs	Program ran successfully	
3	Input necessary information (STV, 2 seats)	N/A	Inputs accepted	Inputs accepted	
4	Check results in Election_Results.txt	N/A	Results reflect expected winner	Results reflect expected winner	

Post condition(s) for Test: The Text file has been created

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: STV_all_first_votes

Name(s) of Testers: Horoom Bula

Test Description:
This is an stv election where all candidates got one first preference votes before dropping so there will be tiebreakers being handled in ballot order and randomized.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Test.csv will be used

Automated: yes___ no __x_

Results: Pass __X___ Fail_____

Preconditions for Test:
There will be an csv before the test, and line 26 of main must have test.csv as well as line 39

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4)), Ballot(2,(2,3,4,1)), Ballot(3,(3,4,1,2)), Ballot(4,(4,1,2,3))	Candidates 1 and 2 will win due to first ballot order	Candidates 1 and 2 will win due to first ballot order	STV same amount of votes test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV election	Inputs accepted	Inputs accepted	
4	Check results in Election_Results.txt	N/A	Results reflect expected winners	Results reflect expected winners	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team#01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: Seats_match_candidates_Pl

Name(s) of Testers: Horoom Bula

Test Description: Seats matches number of candidates for plurality

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __x_

Results: Pass __X__ Fail_____

Preconditions for Test:
There will be an csv before the test, and line 26 of main must have Ptest.csv as well as line 42

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input test data into PTest.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,0,0,0)), Ballot(2,(0,0,0,1)), Ballot(3,(0,0,1,0)), Ballot(4,(0,1,0,0))	Everyone is elected	Everyone is elected	
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	Plurality election, 4 seats	Inputs accepted	Inputs accepted	

4	Check report	Election_Report.txt	Report reflects expected results	Report reflects expected results	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System Team#01

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: Seats_match_candidates_STV

Name(s) of Testers: Horoom Bula

Test Description: Seats matches number of candidates for STV

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __x_

Results: Pass ___X__ Fail_____

Preconditions for Test:
There will be a csv before the test, and line 26 of main must have test.csv as well as line 39

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1					

1	Input test data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4)), Ballot(2,(2,3,4,1)), Ballot(3,(3,4,1,2)), Ballot(4,(4,1,2,3))	Everyone is elected	Everyone is elected	STV seats equal candidates test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV election, 4 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Check results in Election_Results.txt	N/A	Report reflects expected results	Report reflects expected results	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 1st

Test Case ID#: STV_FirstRound

Name(s) of Testers: Caleb Yosef

Test Description:
This is a test where in stv a canidate hits droop in the first round

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Ptest is where it is being stored

Automated: yes___ no _X__

Results: Pass ___x__ Fail_____

Preconditions for Test:
There will be an csv before the test where a candiate can win in the first round, and line 26 of main must have test.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input test data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4)), Ballot(2,(1,2,3,4)), Ballot(3,(3,4,1,2)), Ballot(4,(4,1,2,3))	Candidate 1 and 3 win	Candidate 1 and 3 win	STV exact droop first round test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Check results in Election_Results.txt	N/A	Results reflect expected winners	Results reflect expected winners	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_ **Test Date: November 1st**

Test Case ID#: STVReport **Name(s) of Testers: Caleb Yosef**

Test Description: This is a test that there is a proper report of the STV election

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __X_

Results: Pass ___x__ Fail_____

Preconditions for Test:
There will be an csv of ballots before the test, and line 26 of main must have Test.csv as well as line 39

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4)), Ballot(2,(2,1,3,4)), Ballot(3,(3,4,2,1)), Ballot(4,(1,3,2,4)), Ballot(5,(4,2,1,3))	Report summary appears	Report summary appears	STV election report summary test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Look for summary after election is run		Report summary appears	Report summary appears	

Post condition(s) for Test:

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_ **Test Date: November 1st**

Test Case ID#: PLReport **Name(s) of Testers: Horoom Bula**

Test Description: This is a test that there is a proper report of the plurality election

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no _X__

Results: Pass __x___ Fail_____

Preconditions for Test:
There will be an csv of ballots before the test, and line 26 of main must have Ptest.csv as well as line 42

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into PTest.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,0,0,0)), Ballot(2,(0,1,0,0)), Ballot(3,(0,0,0,1)), Ballot(4,(1,0,0,0)), Ballot(5,(0,0,1,0))	Report summary of the election	Report summary of the election	PL election report summary test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	Plurality, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Look for summary after election is run	N/A	Report summary appears	Report summary appears	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 1st

Test Case ID#: EmptyPL

Name(s) of Testers: Horoom Bula

Test Description: Empty csv for plurality to test waht happens if there are no tests/ candidates

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no __X_

Results: Pass _____ Fail____X____

Preconditions for Test:
There will be an empty csv before the test, and line 26 of main must have Ptest.csv as well as line 42

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: nothing	Election is halted	Election runs	PL empty test
2	Run main program	main() execution	Program blocked	Program runs	
3	Input necessary information	Plurality, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Check for a new text file	N/A	No file generated	Text file created	

Post condition(s) for Test:

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 1st

Test Case ID#: EmptySTV

Name(s) of Testers: Surafiel Ghirmai

Test Description: Empty csv for STV to test waht happens if there are no tests/ candidates

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Test.csv is where it is stored

Automated: yes___ no __X_

Results: Pass _____ Fail____X____

Preconditions for Test:
There will be an empty csv before the test, and line 26 of main must have test.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: nothing	Election is halted	Election goes on	STV empty test
2	Run main program	main() execution	Program blocked	Program runs	
3	Input necessary information	STV, 1 seat, no shuffle	Inputs accepted	Inputs accepted	
4	Check for a new text file	N/A	No file generated	Text file created	

Post condition(s) for Test: test file must not be made

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 1st

Test Case ID#:

Name(s) of Testers: Surafiel Ghirmai

Test Description: Test to see if 10000 ballots can be run before 5 minutes with a report on Plurality

Automated: yes___ no __x_

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Ptest10000.csv is where it is stored

Results: Pass __x___ Fail_____

Preconditions for Test:
There will be an csv with 10000ballot before the test, and line 26 of main must have Ptest10000.csv as well as line 42

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input/use data into PTest10000.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: 10000 ballots	Report is generated in under 5 minutes	Report is generated in under 5 minutes	PL 10,000 ballots performance test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	Plurality, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Check for a new text file	N/A	Text file report generated	Text file report generated	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: STVballot10000

Name(s) of Testers: Surafiel Ghirmai

Test Description: Test to see if 10000 ballots can be run before 5 minutes with a report on stv

Automated: yes___ no _x__

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
test10000.csv is where its stored

Results: Pass __x___ Fail_____

Preconditions for Test:
There will be an csv with 10000 ballots before the test, and line 26 of main must have test10000.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input/use data into Test10000.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: 10000 ballots	Report is generated in under 5 minutes	Report is generated in under 5 minutes	STV 10,000 ballots performance test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV, 2 seats, shuffle	Inputs accepted	Inputs accepted	
4	Check for a new text file	N/A	Text file report generated	Text file report generated	

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: Candidate_duplicateSTV

Name(s) of Testers: Horoom Bula

Test Description: Have a duplicate of the candidates for one ballot

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
test.csv is where its stored

Automated: yes___ no __x_

Results: Pass _____ Fail__x_____

Preconditions for Test:
There will be an csv ballot with a duplicate before the test, and line 26 of main must have test.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1: Ballot(1,(1,2,3,4)), Ballot(2,(2,1,3,4)), Ballot(3,(3,4,2,1)), Ballot(4,(1,3,2,4)), Ballot(5,(4,1,1,3)), Ballot(6,(2,3,4,1)), Ballot(7,(3,1,4,2))	Ballot should be invalid	Election goes on	Candidate invalid ballot test
2	Run main program	main() execution	Program blocked	Program runs	
3	Input necessary information	STV, 2 seats, no shuffle	Inputs accepted	Inputs accepted	
4	Check for a new text file	N/A	Election stops	Election continues	

Post condition(s) for Test: Must be no text file

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#:

Name(s) of Testers: Surafiel Ghirmai

Test Description: A plurality elections is run with 4 candidates and no ballots

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Ptest.csv is where its stored

Automated: yes___ no __x_

Results: Pass _____ Fail____x____

Preconditions for Test:
There will be an empty Ptest.csv before the test, and line 26 of main must have Ptest.csv as well as line 42

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1:	N/A	N/A	Preparing empty ballot test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	Plurality, 2 seats, no shuffle	Electionruns	Election runs on	PL empty test
4	Check for a new txt file	N/A	Text file report created and winner is random	Test file report created and winner is not random	Post condition: report may exist

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#: STV_Empty_Ballot

Name(s) of Testers: Caleb Yosef

Test Description: Having empty ballots for stv with 4 candidates and no ballots

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
test.csv is where its stored

Automated: yes___ no __x_

Results: Pass __x___ Fail_____

Preconditions for Test:
There will be an empty csv ballot before the test, and line 26 of main must have test.csv as well as line 39

Step # 1	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Input data into Test.csv	[Alice, Bob, Charlie, Diana] Test_Case_1:	N/A	N/A	Preparing empty ballot test
2	Run main program	main() execution	Program runs	Program ran successfully	
3	Input necessary information	STV, no seats specified, no shuffle	Random candidate winner	Random candidate winner	STV empty test
4	Check for a new txt file	N/A	Text file report created with randome winner	Text file report created with randome winner	Post condition: report exists

Post condition(s) for Test: The Text file has been created

Project Name: Project 1: Voting System **Team# 01**

Test Stage: Unit __ System _X_

Test Date: November 2nd

Test Case ID#:

Name(s) of Testers: Horoom Bula

Test Description:

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test:

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1					
2					
3					
4					

Post condition(s) for Test:

Project Name: The project #, name of your system, and the team#

Test Stage: Indicate whether it is a unit test or a system test.

Test Date: The date the test was performed.

Test Case ID#: A unique ID is required. Decide on a naming convention and use numbering. Example: Ballot_Shuffle_1

Name(s) of Testers: List the names of anyone involved in running this test case.

Test Description: Describe briefly the test objective.

Automated: Indicate if the test is completely automated or being checked manually. (If you have methods running the tests and checking results, select "yes". If you are manually checking results, indicate manual by selecting the "no.")

Results: Indicate if the test passed or failed.

Step #: You will be listing the test steps in order. This number is the step number in the process.

Test Step Description: Details of the test step.

Test Data: What the test data will be for this step. Be clear on what the input data will be. If using a specific file, be clear on the name.

Expected Result: What result are you expecting from the program component or system.

Actual Result: What result were returned based on the test.

Post condition for Test: What will be true after the test has been run? Has the state of the system changed in any way?

Notes: Comments and notes for you and your team members.